

Lecture 7: Introduction to Formal Methods

CS 580B1: Trustworthy Machine Learning

Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn ([link](#) to their course)

[EECS 219C: Formal Methods: Specification, Verification, and Synthesis](#) at UC Berkeley by Sanjit Seshia

[CS 357 Advanced Topics in Formal Methods](#) at Stanford by Aleksandar Zeljic

Other relevant courses on Trustworthy Machine Learning:

[CS 329T](#) at Stanford

[CS 521](#) at UIUC

[ECE1784H/CSC2559H](#) at U Toronto

Course logistics

- No reviews due this week
- Please start forming project teams and thinking about project ideas
- Use of LLMs for writing reviews

Quick recap

- (Two weeks back) Attack algorithms for finding adversarial examples
- (Last week) Defending against adversarial examples via training-time and inference-time techniques
- (This week) Introduction to formal methods
- (Next week) Certifying local robustness of DNNs using formal methods

What is a proof?

What is a proof?

- Sequence of steps from assumptions to conclusion convincing to a skeptic
- **What is the correct step size?**
Small vs. Big vs. **Gap**
- Computers can **only perform small steps.**

What is a formal proof?

What is a formal proof?

- Sequence of steps from assumptions to conclusion convincing to a **computer**

- **What is the correct step size?**

Small

- Computers can **only perform small steps.**

Formal proof system

- Axioms
 - Reflexivity: $\forall x. x = x$
 - Symmetry: $\forall x, y. x = y \Rightarrow y = x$
 - Transitivity: $\forall x, y, z. x = y \wedge y = z \Rightarrow x = z$
- Inference rules:

$$A \quad A \Rightarrow B$$

$$B$$

Search for a proof!

Formal methods

Computational methods for verifying
and searching for formal proofs.

Applications of formal methods

MATHEMATICS

Maths proof smashes size record

Supercomputer produces a 200-terabyte proof — but is it really mathematics?

BY EVELYN LAMB

Computer-generated proof of the PFR conjecture is the largest ever

Mathematics

Mathematicians plan computer proof of Fermat's last theorem

Fermat's last theorem puzzled mathematicians for centuries until it was finally proven in 1993. Now, researchers want to create a version of the proof that can be formally checked by a computer for any errors in logic

By Alex Wilkins

📅 18 March 2024



Terence Tao

@tao

🌐 Nov 21, 2023 *

The #Lean4 formalization of the proof of the PFR conjecture has uncovered an extremely minor error in the paper itself; see github.com/teorth/pfr/pull/53. The issue is that in Lemma 5.2, the hypothesis that G is a vector space over F_2 (which is actually true throughout the paper) was omitted in the statement of this lemma, but is a needed hypothesis (in the proof one wants to rewrite $X - Y - Z$ as $X + Y + Z$, and this is only possible in characteristic 2). Thanks to Floris van Doorn for uncovering this issue. The fix is immediate (just put this omitted hypothesis back in), but it does show (for the second time, in my case) that formalization can uncover (small) errors in the original proof being formalized.

Applications of formal methods

Formal verification of a realistic compiler

Xavier Leroy
INRIA Paris-Rocquencourt
Domaine de Voluceau, B.P. 105, 78153 Le Chesnay, France
xavier.leroy@inria.fr

Formal reasoning about the security of Amazon Web Services

Byron Cook
Amazon Web Services &
University College London

MACHINE LEARNING

How to Guarantee the Safety of Autonomous Vehicles



As computer-driven cars and planes become more common, the key to preventing accidents, researchers show, is to know what you don't know.

Formal methods applied to computational systems

Verification: Does the system do what its supposed to do?



Our focus

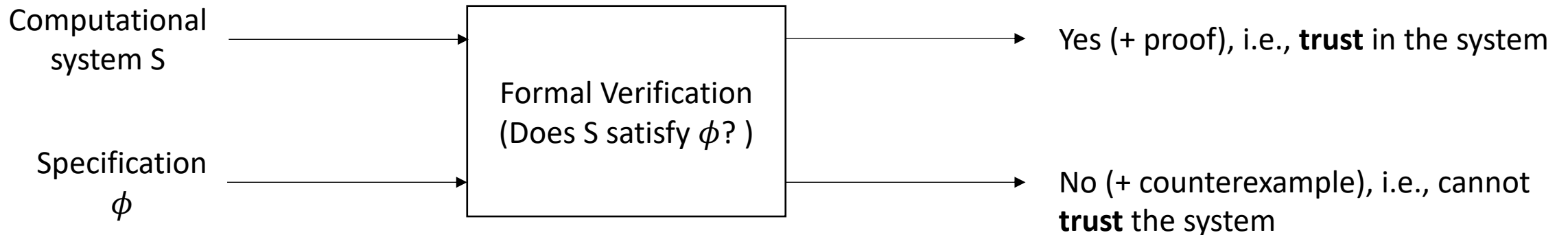
Synthesis: How can a system be constructed to do what its supposed to do?

Formal verification

Does a computational system do what its supposed to do?



Does the mathematical model of a computational system satisfy a mathematical specification?



Key aspects of formal verification

Formal methods $\approx$ Computational Proof methods

Specification:

- What must the system (program) do?
- Includes building mathematical models of systems

Specification $\approx$ Statement of Conjecture/Theorem

Verification:

- Does the system do what its supposed to do?

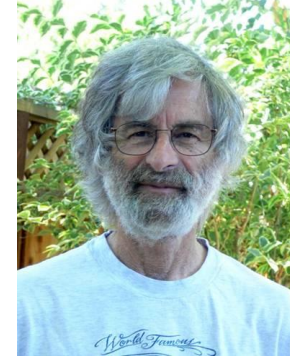
Verification $\approx$ Proving/Disproving the Conjecture



An axiomatic basis for computer programming

Tony Hoare; *CACM* 1969

In this paper an attempt is made to explore the logical foundations of computer programming by use of techniques which were first applied in the study of geometry and have later been extended to other branches of mathematics. This involves the elucidation of sets of axioms and rules of inference which can be used in proofs of the properties of computer programs. Examples are given of such axioms and rules, and a formal proof of a simple theorem is displayed. Finally, it is argued that important advantage, both theoretical and practical, may follow from a pursuance of these topics.



Who builds a house without drawing blueprints ? Leslie Lamport; CACM 2015

Architects draw detailed plans before a brick is laid or a nail is hammered. But few programmers write even a rough sketch of what their programs will do before they start coding. We can learn from architects. A blueprint for a program is called a *specification*. An architect's blueprint is a useful metaphor for a software specification.

The need for specifications follows from two observations. The first is that it is a good idea to think about what we are going to do before doing it, and as the cartoonist Guindon wrote: "Writing is nature's way of letting you know how sloppy your thinking is." The second observation is that to write a good program, we need to think above the code level.

The main reason for writing a formal spec is to apply tools to check it.
Tools cannot find design errors in informal specifications.

Approaches to formal verification

- Automated verification:

- Model checking
- Abstract interpretation
- Type checking
- Testing
- ...

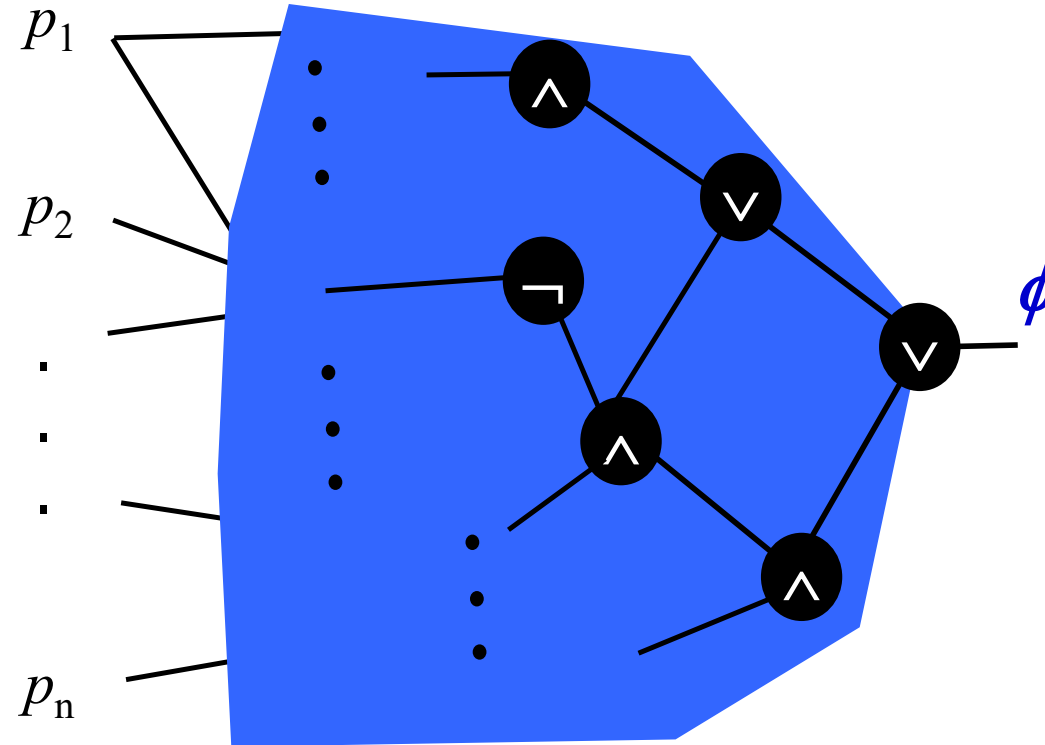
- Interactive theorem proving



Frequently use constraint solvers
such as SAT/SMT solvers

- In general, formal verification is an undecidable problem!

Boolean satisfiability (SAT)

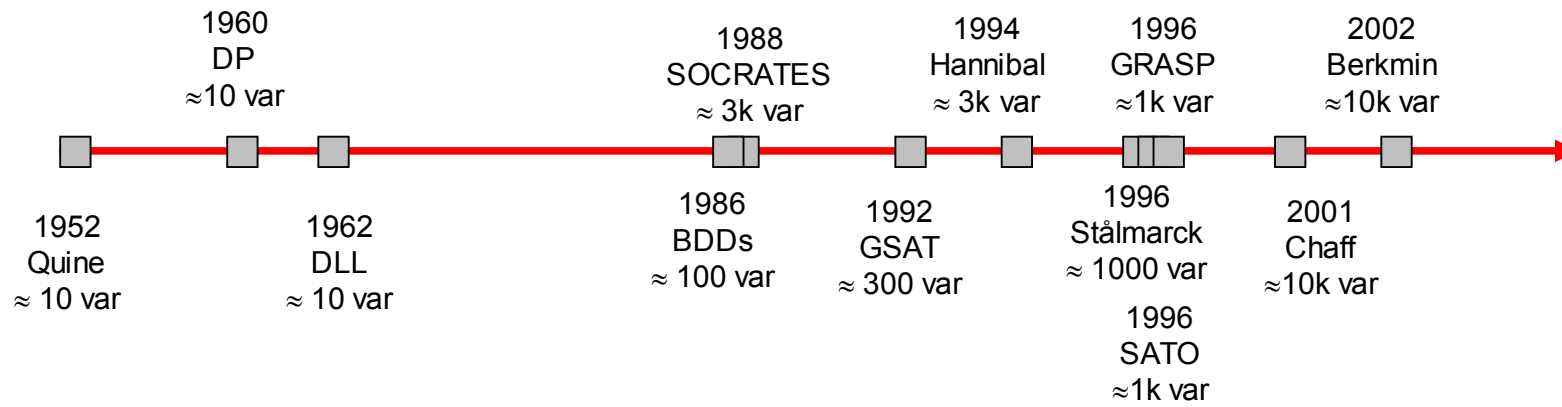


**Is there an assignment to the p_i variables
s.t. ϕ evaluates to 1?**

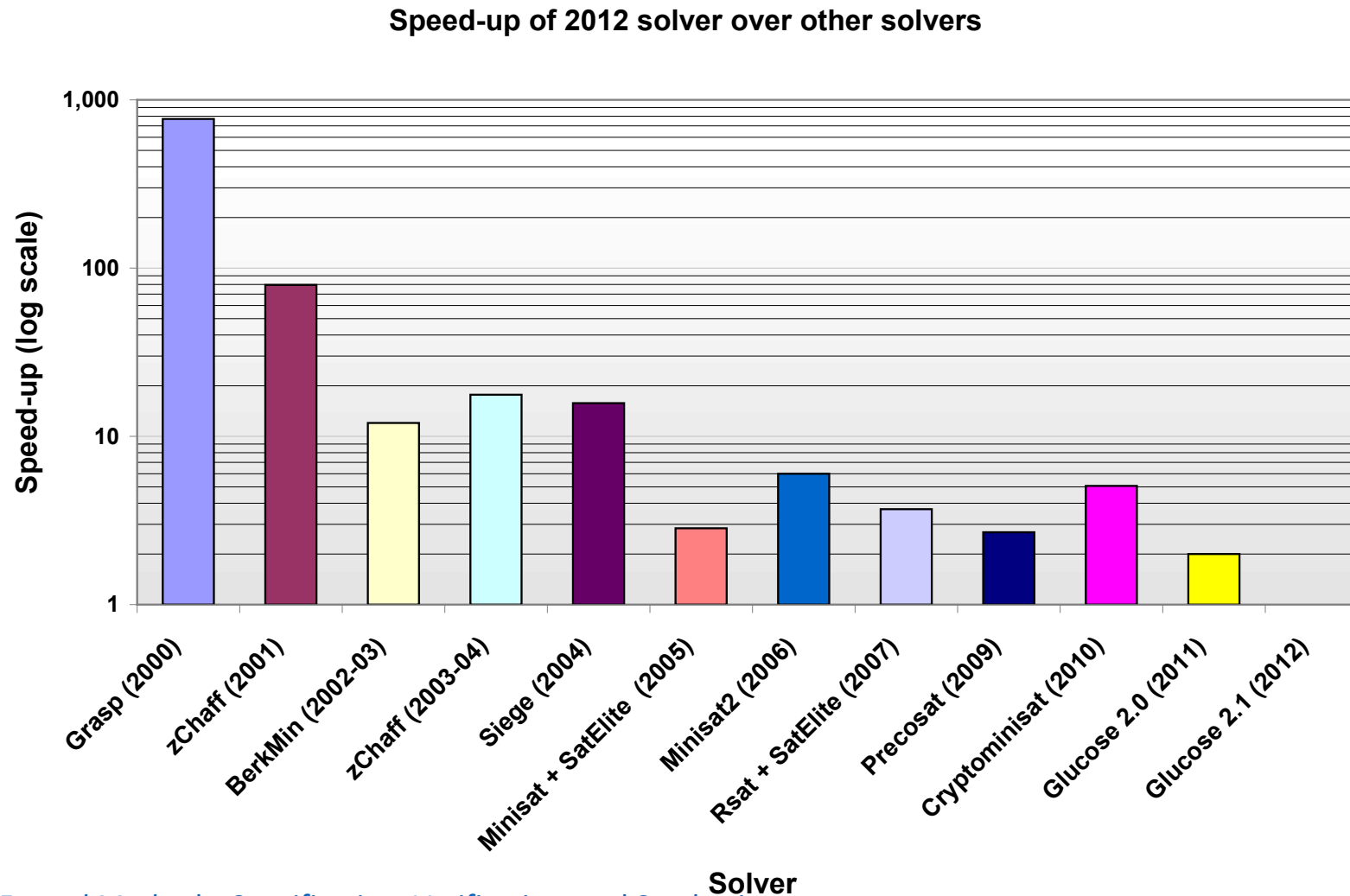
Boolean satisfiability (SAT)

Propositional Satisfiability: Given a formula over Boolean variables, is there an assignment of 0/1's to variables which makes the formula true

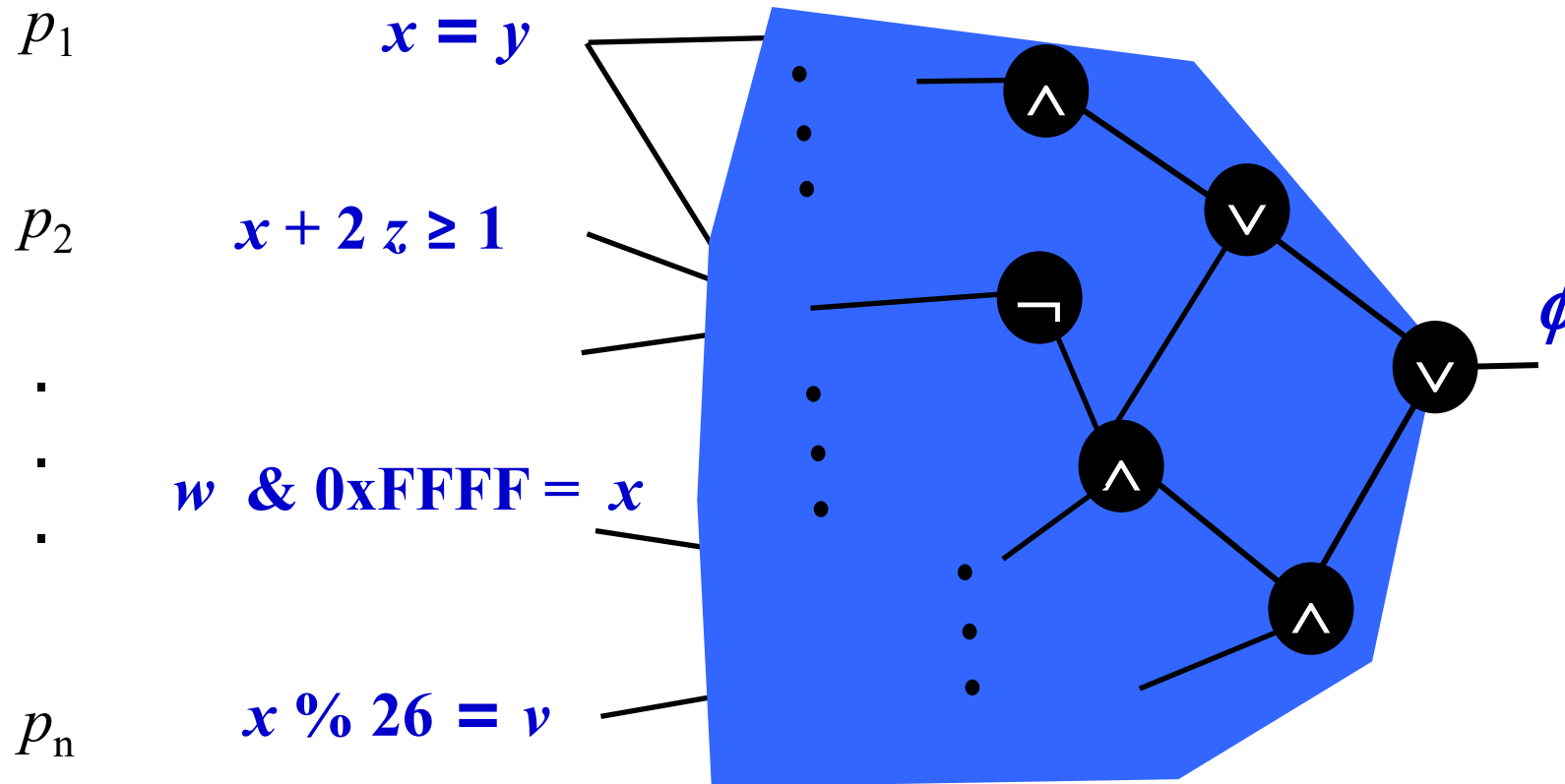
- Canonical NP-hard problem (Cook 1973)
- Enormous progress in tools that can solve instances with thousands of variables and millions of clauses
- Also at the core of SMT solvers (extensions to richer classes of constraints)



SAT improvements



Satisfiability Modulo Theory (SMT)



**Is there an assignment to the x, y, z, w variables
s.t. ϕ evaluates to 1?**

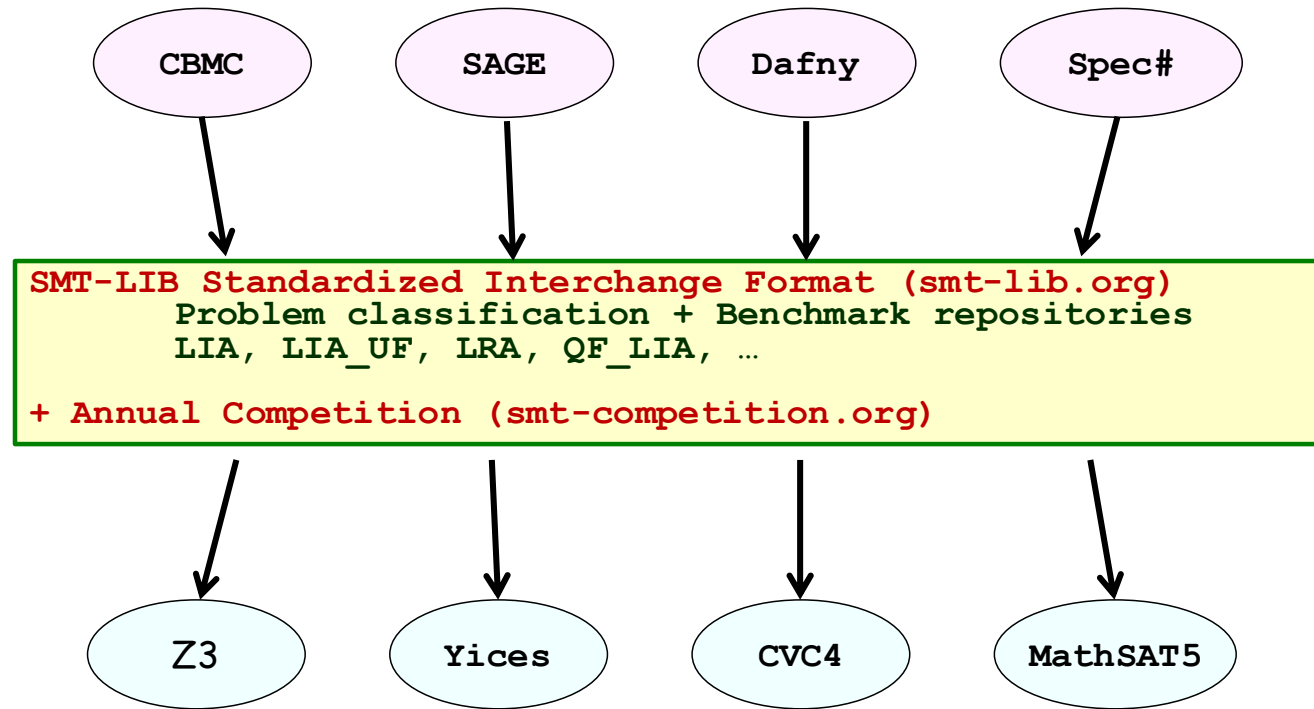
Satisfiability Modulo Theory (SMT)

- ❑ Computational problem: Find a satisfying assignment to a formula
 - Boolean + Int types, logical connectives, arithmetic operators
 - Bit-vectors + bit-manipulation operations in C
 - Boolean + Int types, logical/arithmetic ops + Uninterpreted functions
- ❑ “Modulo Theory”: Interpretation for symbols is fixed
 - Can use specialized algorithms (e.g. for arithmetic constraints)
- ❑ Progress in improved SMT solvers

Little Engines of Proof

SAT; Linear arithmetic; Congruence closure

Satisfiability Modulo Theory (SMT)



Ever-growing scalability and use in different applications

Constraint satisfaction problem: Example 1

x, y, z : Boolean variables

Find values such that all the following constraints are satisfied

$$x \mid \sim y \mid \sim z$$

$$\sim x \mid y \mid \sim z$$

$$\sim x \mid \sim y \mid z$$

Constraint satisfaction problem: Example 2

x, y, z : real-valued variables

Find values such that all the following constraints are satisfied

$$2x + 3y - 5z \leq 7$$

$$-5x + 2y \leq 106$$

$$9x - 4y + 29z \leq -1$$

What if we change the type to "integer" variables

Constraint satisfaction problem: Example 3

x, y, z : real-valued variables

Find values such that all the following constraints are satisfied

$$2x + 3y - 5z \leq 7 \mid 3x - 4z \leq 9$$

$$-5x + 2y \leq 106 \mid 6y - 78z \leq 567$$

$$9x - 4y + 29z \leq -1 \mid 5x - 7y + z \leq 98$$

Mixture of Boolean and linear constraints

Solving Sudoku as constraint satisfaction

					3		8	5
		1		2				
			5		7			
		4				1		
	9							
5							7	3
		2		1				
				4				9

Solving Sudoku as constraint satisfaction

					3		8	5
		1		2				
			5		7			
		4				1		
	9							
5							7	3
		2		1				
				4				9

Variables: $x(i,j)$, for $i=1..9$ and $j=1..9$, of integer value between 1 and 9
Find values such that all the following constraints are satisfied

Some values are known:

$x(3,3)=1, x(2,6)=3, \dots$

Every value appears in each row:

for each $k=1, \dots, 9$:

for each $i=1, \dots, 9$:

$x(i,1)=k \mid x(i,2)=k \mid \dots \mid x(i,9)=k$

Every value appears in each column: similar constraints

Every value appears in each square:

for each $k = 1, \dots, 9$,

$x(1,1)=k \mid x(1,2)=k \mid x(1,3)=k \mid \dots \mid x(3,2)=k \mid x(3,3)=k$